

SQL Tuning Case Study

Solution : Index

Index Tuning

INDEX CASE 1

다음의 SELECT 문장을 실행하고 실행계획을 확인한 후 문제점을 진단하고 해결책을 제시합니다.

```
SQL> select /*+ index(emp2 emp2_x01) */ *
      from emp2
      where deptno=30 and sal >=2000;
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		8192	00:00:00.33	2338
* 1	TABLE ACCESS BY INDEX ROWID BATCHED	EMP2	1	30492	8192	00:00:00.33	2338
* 2	INDEX RANGE SCAN	EMP2_X01	1	38229	49152	00:00:00.14	238

Predicate Information (identified by operation id):

```
1 - filter("SAL">=2000)
2 - access("DEPTNO"=30)
```

```
SQL> @idx
tab_name의 값을 입력하십시오: emp2
```

INDEX_NAME	INDEX_TYPE	UNIQUENESS	COLUMNS
EMP2_X01	NORMAL	NONUNIQUE	DEPTNO+JOB

1개의 행이 선택되었습니다.

문제점)

Access 단계로 인덱스를 탐색해서 49152 건 데이터의 rowid를 알아내서 테이블을 49152 번 랜덤 액세스합니다. 그러나 랜덤 액세스에서 8192 건으로 필터링 되고 있습니다.

성능 부하의 원인이 되는 랜덤 액세스를 줄이는 것이 필요합니다.

해결)

액세스 조건절의 deptno 컬럼과 필터링 조건으로 사용되었던 sal 컬럼을 하나의 인덱스로 구성하여 인덱스에서 모든 조건을 다 체크할 수 있게 합니다.

```
SQL> create index emp2_x02 on emp2(deptno, sal) nologging;
```

```
SQL> select /*+ index(emp2 emp2_x02) */ *
        from emp2
        where deptno=30 and sal >=2000;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads
0	SELECT STATEMENT		1		8192	00:00:00.01	854	22
1	TABLE ACCESS BY INDEX ROWID	EMP2	1	30492	8192	00:00:00.01	854	22
* 2	INDEX RANGE SCAN	EMP2_X02	1	30492	8192	00:00:00.01	104	22

Predicate Information (identified by operation id):

```
2 - access("DEPTNO"=30 AND "SAL">=2000 AND "SAL" IS NOT NULL)
```

```
SQL> drop index emp2_x02;
```

또는 기존의 인덱스에 필터 컬럼을 추가합니다.

```
SQL> create index emp2_x02 on emp2(deptno, job, sal) nologging;
```

```
SQL> select /*+ index(emp2 emp2_x02) */ *
        from emp2
        where deptno=30 and sal >=2000;
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads
0	SELECT STATEMENT		1		8192	00:00:00.01	1015	181
1	TABLE ACCESS BY INDEX ROWID	EMP2	1	30492	8192	00:00:00.01	1015	181
* 2	INDEX RANGE SCAN	EMP2_X02	1	30492	8192	00:00:00.01	265	181

Predicate Information (identified by operation id):

```
2 - access("DEPTNO"=30 AND "SAL">=2000)
    filter("SAL">=2000)
```

INDEX CASE 2

다음 문장의 실행 계획을 통해 문제점을 식별하고 성능을 향상시키시오.
필요한 인덱스를 생성하시요.

```
SQL> SELECT cust_id, cust_last_name, cust_year_of_birth, cust_city, country_id
      FROM   custs c
      WHERE  cust_credit_limit BETWEEN 1500 AND 3000
      AND    cust_year_of_birth = 1990 ;
```

```
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		3	00:00:00.01	1141
* 1	TABLE ACCESS FULL	CUSTS	1	175	3	00:00:00.01	1141

Predicate Information (identified by operation id):

```
1 - filter(("CUST_YEAR_OF_BIRTH"=1990 AND "CUST_CREDIT_LIMIT"<=3000 AND
          "CUST_CREDIT_LIMIT">=1500))
```

해결 1) 필터링 조건이 좋은 단일 컬럼의 인덱스를 이용

해결 2) 결합 인덱스를 이용

해결 1) 필터링 조건이 좋은 단일 컬럼의 인덱스를 이용

```
SQL> @idx
tab_name의 값을 입력하십시오: custs
```

INDEX_NAME	INDEX_TYPE	UNIQUENESS	COLUMNS
CUST_IX01	NORMAL	UNIQUE	CUST_ID

```
SQL> select count(case when cust_credit_limit between 1500 and 3000 then 1 end) as limit,
       count(case when cust_year_of_birth=1990 then 1 end) as birth
       from custs;
```

LIMIT	BIRTH
19309	31

CUST_YEAR_OF_BIRTH 컬럼에 대한 조건이 필터링이 좋습니다. 따라서 문제의 명령어만 고려한다면 CUST_YEAR_OF_BIRTH 컬럼에 대한 인덱스가 필터링이 좋은 단일 인덱스가 될 수 있습니다.

```
SQL> create index custs_credit_ix on custs(cust_credit_limit) nologging;
```

인덱스가 생성되었습니다.

```
SQL> create index custs_birth_ix on custs(cust_year_of_birth) nologging;
```

인덱스가 생성되었습니다.

```
SQL> select /*+ index(c(cust_credit_limit)) */
       cust_id, cust_last_name, cust_year_of_birth, cust_city, country_id
       from custs c
       where cust_credit_limit between 1500 and 3000
       and cust_year_of_birth=1990;
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		3	00:00:00.07	2090
* 1	TABLE ACCESS BY INDEX ROWID BATCHED	CUSTS	1	175	3	00:00:00.07	2090
* 2	INDEX RANGE SCAN	CUSTS_CREDIT_IX	1	13104	19309	00:00:00.03	40

Predicate Information (identified by operation id):

- ```
1 - filter("CUST_YEAR_OF_BIRTH"=1990)
2 - access("CUST_CREDIT_LIMIT">=1500 AND "CUST_CREDIT_LIMIT"<=3000)
```

```
SQL> select /*+ index(c(cust_year_of_birth)) */
 cust_id, cust_last_name, cust_year_of_birth, cust_city, country_id
 from custs c
 where cust_credit_limit between 1500 and 3000
 and cust_year_of_birth=1990
```

| Id  | Operation                   | Name           | Starts | E-Rows | A-Rows | A-Time      | Buffers | Reads |
|-----|-----------------------------|----------------|--------|--------|--------|-------------|---------|-------|
| 0   | SELECT STATEMENT            |                | 1      |        | 3      | 00:00:00.01 | 32      | 1     |
| * 1 | TABLE ACCESS BY INDEX ROWID | CUSTS          | 1      | 175    | 3      | 00:00:00.01 | 32      | 1     |
| * 2 | INDEX RANGE SCAN            | CUSTS_BIRTH_IX | 1      | 740    | 31     | 00:00:00.01 | 3       | 1     |

Predicate Information (identified by operation id):

```
1 - filter(("CUST_CREDIT_LIMIT"<=3000 AND "CUST_CREDIT_LIMIT">=1500))
2 - access("CUST_YEAR_OF_BIRTH"=1990)
```

```
SQL> drop index custs_credit_ix;
```

인덱스가 삭제되었습니다.

```
SQL> drop index custs_birth_ix;
```

인덱스가 삭제되었습니다

해결 2) 결합 인덱스를 이용

```
SQL> create index custs_ix02 on custs(cust_year_of_birth, cust_credit_limit) nologging;
```

인덱스가 생성되었습니다.

```
SQL> SELECT /*+ index(c custs_ix02)) */
 cust_id, cust_last_name, cust_year_of_birth, cust_city, country_id
 FROM custs c
 WHERE cust_credit_limit BETWEEN 1500 AND 3000
 AND cust_year_of_birth = 1990
```

| Id  | Operation                   | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|-----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT            |            | 1      |        | 3      | 00:00:00.01 | 6       |
| 1   | TABLE ACCESS BY INDEX ROWID | CUSTS      | 1      | 3      | 3      | 00:00:00.01 | 6       |
| * 2 | INDEX RANGE SCAN            | CUSTS_IX02 | 1      | 3      | 3      | 00:00:00.01 | 3       |

Predicate Information (identified by operation id):

```
2 - access("CUST_YEAR_OF_BIRTH"=1990 AND "CUST_CREDIT_LIMIT">=1500 AND
 "CUST_CREDIT_LIMIT"<=3000)
```

조건절의 컬럼들로 구성된 결합인덱스는 랜덤엑세스를 최소화할 수 있어서 가장 최적화된 실행계획을 구성할 수 있습니다.

```
SQL> drop index custs_ix02;
```

인덱스가 삭제되었습니다.

## INDEX CASE 3

다음의 문장이 인덱스를 이용하여 정렬 작업을 대체할 수 있도록 필요한 사항을 구현하시오.

```
SQL> SELECT *
 FROM emp
 ORDER BY sal ASC ;
```

```
SQL> @xp1an
```

| Id | Operation         | Name | Starts | E-Rows | A-Rows | A-Time      | Buffers | OMem | 1Mem | Used-Mem |
|----|-------------------|------|--------|--------|--------|-------------|---------|------|------|----------|
| 0  | SELECT STATEMENT  |      | 1      |        | 14     | 00:00:00.01 | 9       |      |      |          |
| 1  | SORT ORDER BY     |      | 1      | 14     | 14     | 00:00:00.01 | 9       | 2048 | 2048 | 2048 (0) |
| 2  | TABLE ACCESS FULL | EMP  | 1      | 14     | 14     | 00:00:00.01 | 9       |      |      |          |

```
SQL> desc emp
```

| Name     | Null?    | Type         |
|----------|----------|--------------|
| EMPNO    | NOT NULL | NUMBER(4)    |
| ENAME    |          | VARCHAR2(10) |
| JOB      |          | VARCHAR2(9)  |
| MGR      |          | NUMBER(4)    |
| HIREDATE |          | DATE         |
| SAL      |          | NUMBER(7,2)  |
| COMM     |          | NUMBER(7,2)  |
| DEPTNO   |          | NUMBER(2)    |

```
SQL> @idx
```

Enter value for tab\_name: emp

| INDEX_NAME    | INDEX_TYPE | UNIQUENESS | COLUMNS  |
|---------------|------------|------------|----------|
| EMP_JOB_IX    | NORMAL     | NONUNIQUE  | JOB      |
| EMP_MGR_IX    | NORMAL     | NONUNIQUE  | MGR      |
| EMP_SAL_IX    | NORMAL     | NONUNIQUE  | SAL      |
| EMP_COMM_IX   | NORMAL     | NONUNIQUE  | COMM     |
| EMP_HIRE_IX   | NORMAL     | NONUNIQUE  | HIREDATE |
| EMP_EMPNO_IX  | NORMAL     | UNIQUE     | EMPNO    |
| EMP_ENAME_IX  | NORMAL     | NONUNIQUE  | ENAME    |
| EMP_DEPTNO_IX | NORMAL     | NONUNIQUE  | DEPTNO   |

문제점)

SAL 컬럼에 인덱스가 있으나 NULL 값을 포함할 수 있으므로 정렬 작업 대신 사용할 수 없습니다. NOT NULL 제약조건을 선언하거나 조건절을 추가하여 NULL값을 배제할 수 있도록 합니다.

- 정렬 작업을 수행할 컬럼에 인덱스 확인 (필요시 생성)
- 모든 행이 만족할 수 있는 조건식 추가 또는 NOT NULL 제약 조건 추가



```
SQL> SELECT /*+ index(emp(sal)) */ *
 FROM emp
 ORDER BY sal ASC ;
SQL> @xplan
```

| Id | Operation         | Name | Starts | E-Rows | A-Rows | A-Time      | Buffers | OMem | 1Mem | Used-Mem |
|----|-------------------|------|--------|--------|--------|-------------|---------|------|------|----------|
| 0  | SELECT STATEMENT  |      | 1      |        | 14     | 00:00:00.01 | 9       |      |      |          |
| 1  | SORT ORDER BY     |      | 1      | 14     | 14     | 00:00:00.01 | 9       | 2048 | 2048 | 2048 (0) |
| 2  | TABLE ACCESS FULL | EMP  | 1      | 14     | 14     | 00:00:00.01 | 9       |      |      |          |

조건식이 없는 상황에서 힌트를 이용한 인덱스 사용 유도는 반영되지 않습니다.

(NULL 의 존재 가능성 때문)

```
SQL> SELECT *
 FROM emp
 WHERE sal IS NOT NULL
 ORDER BY sal ASC;
SQL> @xplan
```

| Id  | Operation                   | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|-----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT            |            | 1      |        | 14     | 00:00:00.01 | 4       |
| 1   | TABLE ACCESS BY INDEX ROWID | EMP        | 1      | 14     | 14     | 00:00:00.01 | 4       |
| * 2 | INDEX FULL SCAN             | EMP_SAL_IX | 1      | 14     | 14     | 00:00:00.01 | 2       |

Predicate Information (identified by operation id):

2 - filter("SAL" IS NOT NULL)

## INDEX CASE 4

다음 문장의 성능을 확인하고 인덱스의 사용을 최적화 가능하도록 필요한 사항을 구현하시오.

```
SQL> @C:\labs\test\mcustsum.sql -- mcustsum 테이블과 관련 인덱스를 생성
SQL> SELECT COUNT(*)
 FROM mcustsum t
 WHERE salegb = 'A'
 AND salemm BETWEEN '200801' AND '200812' ;
```

SQL>@xplan

| Id  | Operation            | Name      | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------|-----------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT     |           | 1      |        | 1      | 00:00:00.03 | 2622    |
| 1   | SORT AGGREGATE       |           | 1      | 1      | 1      | 00:00:00.03 | 2622    |
| * 2 | INDEX FAST FULL SCAN | MCUST_X01 | 1      | 456K   | 76084  | 00:00:00.03 | 2622    |

Predicate Information (identified by operation id):

```
2 - filter(("SALEGB"='A' AND "SALEMM">='200801' AND "SALEMM"<='200812'))
```

문제점) 결합 인덱스의 선행 컬럼의 조건의 범위 검색이어서 인덱스 검색의 범위를 줄여주지 못하고 INDEX FAST FULL SCAN으로 실행되었습니다.

SQL> desc mcustsum

| Name    | Null? | Type         |
|---------|-------|--------------|
| CUSTNO  |       | NUMBER       |
| SALEMM  |       | VARCHAR2(12) |
| SALEGB  |       | VARCHAR2(1)  |
| SALEAMT |       | NUMBER       |

SQL> @idx

Enter value for tab\_name: mcustsum

| INDEX_NAME | INDEX_TYPE | UNIQUENESS | COLUMNS       |
|------------|------------|------------|---------------|
| MCUST_X01  | NORMAL     | NONUNIQUE  | SALEMM+SALEGB |

```
SQL> select table_name, num_rows
 from user_tab_statistics
 where table_name='MCUSTSUM';
```

| TABLE_NAME | NUM_ROWS |
|------------|----------|
| MCUSTSUM   | 913004   |

```
SQL> select table_name, column_name, num_distinct, density
 from user_tab_col_statistics
 where table_name='MCUSTSUM';
```

| TABLE_NAME | COLUMN_NAME | NUM_DISTINCT | DENSITY    |
|------------|-------------|--------------|------------|
| MCUSTSUM   | CUSTNO      | 913004       | 1.0953E-06 |
| MCUSTSUM   | SALEMM      | 10           | .1         |
| MCUSTSUM   | SALEGB      | 2            | .5         |
| MCUSTSUM   | SALEAMT     | 991          | .001009082 |

해결 1) 점 조건식을 사용하는 컬럼을 선행 컬럼으로 인덱스 재구성

```
SQL> create index mcust_x02 on mcustsum(salegb,salemm) nologging;
```

인덱스가 생성되었습니다.

```
SQL> SELECT /*+ index(t mcust_x02) */ COUNT(*)
 FROM mcustsum t
 WHERE salegb = 'A'
 AND salemm BETWEEN '200801' AND '200812' ;
```

| Id  | Operation        | Name      | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|------------------|-----------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT |           | 1      |        | 1      | 00:00:00.01 | 214     |
| 1   | SORT AGGREGATE   |           | 1      | 1      | 1      | 00:00:00.01 | 214     |
| * 2 | INDEX RANGE SCAN | MCUST_X02 | 1      | 456K   | 76084  | 00:00:00.01 | 214     |

Predicate Information (identified by operation id):

```
2 - access("SALEGB"='A' AND "SALEMM">='200801' AND "SALEMM"<='200812')
```

```
SQL> drop index mcust_x02;
```

인덱스가 삭제되었습니다.

해결 2) 선분 조건식( >, <, BETWEEN, LIKE 등)을 점 조건식(=,IN) 으로 변경

```
SQL> SELECT COUNT(*) AS cnt,
 2 COUNT(CASE WHEN salegb = 'A' THEN 1 END) AS salegb,
 3 COUNT(CASE WHEN salemm BETWEEN '200801' AND '200812' THEN 1 END) AS salemm
 4 FROM mcustsum ;
```

| CNT    | SALEGB | SALEMM |
|--------|--------|--------|
| 913004 | 76084  | 913004 |

SALEGB 컬럼이 SALEMM 컬럼보다 필터링 조건이 뛰어납니다. 하지만 MCUST\_X01 인덱스는 SALEMM 컬럼이 선행으로 들어있고 조건식이 선분 조건을 가지므로 스캔 범위가 넓습니다. 이러한 선분 조건을 등가 조건식(점 조건식)으로 변경하면 각 등가 조건에 해당되는 영역만 별도로 스캔이 가능합니다.

```
SQL> SELECT /*+ index(t mcust_x01) */ COUNT(*)
 FROM mcustsum t
 WHERE salegb = 'A'
 AND salemm IN ('200801', '200802', '200803', '200804', '200805',
 '200806', '200807', '200808', '200809', '200810',
 '200811', '200812') ;
```

```
SQL> @xp1an
```

| Id  | Operation        | Name      | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|------------------|-----------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT |           | 1      |        | 1      | 00:00:00.01 | 237     |
| 1   | SORT AGGREGATE   |           | 1      | 1      | 1      | 00:00:00.01 | 237     |
| 2   | INLIST ITERATOR  |           | 1      |        | 76084  | 00:00:00.01 | 237     |
| * 3 | INDEX RANGE SCAN | MCUST_X01 | 12     | 456K   | 76084  | 00:00:00.01 | 237     |

Predicate Information (identified by operation id):

```
3 - access((("SALEMM"='200801' OR "SALEMM"='200802' OR "SALEMM"='200803' OR
"SALEMM"='200804' OR "SALEMM"='200805' OR "SALEMM"='200806' OR "SALEMM"='200807'
OR "SALEMM"='200808' OR "SALEMM"='200809' OR "SALEMM"='200810' OR
"SALEMM"='200811' OR "SALEMM"='200812')) AND "SALEGB"='A')
```

INLIST 실행 계획은 각각의 등가 조건이 별도의 시작점으로 스캔하므로 스캔 범위를 좁힐 수 있습니다.

## 참고) SALEGB 컬럼의 분포에 대한 통계 수집

```
SQL> select table_name, column_name, num_distinct, density, histogram
 from user_tab_col_statistics
 where table_name='MCUSTSUM';
```

| TABLE_NAME | COLUMN_NAM | NUM_DISTINCT | DENSITY    | HISTOGRAM |
|------------|------------|--------------|------------|-----------|
| MCUSTSUM   | CUSTNO     | 913004       | 1.0953E-06 | NONE      |
| MCUSTSUM   | SALEMM     | 10           | .1         | NONE      |
| MCUSTSUM   | SALEGB     | 2            | 5.5115E-07 | NONE      |
| MCUSTSUM   | SALEAMT    | 991          | .001009082 | NONE      |

## SALEGB 컬럼의 분포에 대한 통계 수집

```
SQL> begin
 dbms_stats.gather_table_stats('test','mcustsum',method_opt=> 'for columns salegb size auto');
 end;
 /
```

```
SQL> select table_name, column_name, num_distinct, density, histogram
 from user_tab_col_statistics
 where table_name='MCUSTSUM';
```

| TABLE_NAME | COLUMN_NAM | NUM_DISTINCT | DENSITY    | HISTOGRAM |
|------------|------------|--------------|------------|-----------|
| MCUSTSUM   | CUSTNO     | 913004       | 1.0953E-06 | NONE      |
| MCUSTSUM   | SALEMM     | 10           | .1         | NONE      |
| MCUSTSUM   | SALEGB     | 2            | 5.5115E-07 | FREQUENCY |
| MCUSTSUM   | SALEAMT    | 991          | .001009082 | NONE      |

```
SQL> SELECT COUNT(*)
 FROM mcustsum t
 WHERE salegb = 'A'
 AND salemm BETWEEN '200801' AND '200812' ;
```

```
SQL> @xplan
```

| Id  | Operation            | Name      | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------|-----------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT     |           | 1      |        | 1      | 00:00:00.02 | 2622    |
| 1   | SORT AGGREGATE       |           | 1      | 1      | 1      | 00:00:00.02 | 2622    |
| * 2 | INDEX FAST FULL SCAN | MCUST_X01 | 1      | 74845  | 76084  | 00:00:00.02 | 2622    |

Predicate Information (identified by operation id):

```
2 - filter(("SALEGB"='A' AND "SALEMM">='200801' AND "SALEMM"<='200812'))
<- 예측치가 정확해짐
```

```
SQL> create index mcust_x02 on mcustsum(salegb,salemm) nologging;
```

```
SQL> SELECT COUNT(*)
 FROM mcustsum t
 WHERE salegb = 'A'
 AND salemm BETWEEN '200801' AND '200812' ;
```

```
SQL> @xplan
```

| Id  | Operation        | Name      | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|------------------|-----------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT |           | 1      |        | 1      | 00:00:00.01 | 214     |
| 1   | SORT AGGREGATE   |           | 1      | 1      | 1      | 00:00:00.01 | 214     |
| * 2 | INDEX RANGE SCAN | MCUST_X02 | 1      | 74845  | 76084  | 00:00:00.01 | 214     |

Predicate Information (identified by operation id):

```
2 - access("SALEGB"='A' AND "SALEMM">='200801' AND "SALEMM"<='200812')
```

<- 예측이 정확해지면서 힌트가 없어도 인덱스를 골라 사용할 수 있음

```
SQL> drop index mcust_x02;
```

## INDEX CASE 5

다음 문장의 실행계획을 확인하고 인덱스를 최대한 활용할 수 있도록 필요한 사항을 구현하시오.

```
SQL> SELECT channel_id, MAX(time_id)
 FROM sales
 WHERE channel_id = 3
 GROUP BY channel_id;
```

```
CHANNEL_ID MAX(TIME

 3 01/12/31
```

```
SQL>@explain
```

| Id  | Operation            | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT     |            | 1      |        | 1      | 00:00:00.14 | 4287    |
| 1   | SORT GROUP BY NOSORT |            | 1      | 536K   | 1      | 00:00:00.14 | 4287    |
| * 2 | INDEX FAST FULL SCAN | SALES_IX01 | 1      | 536K   | 536K   | 00:00:00.07 | 4287    |

Predicate Information (identified by operation id):

```
2 - filter("CHANNEL_ID"=3)
```

문제점)

최적의 인덱스가 없어서 Full Table Scan 대신 그나마 크기가 작은 인덱스를 FAST FULL SCAN으로 탐색하여 실행하고 있습니다. 그러나 사용한 인덱스의 크기가 커서 성능의 최적화가 되지 않고 있습니다.

```
SQL> @idx
Enter value for tab_name: sales
```

| INDEX_NAME | INDEX_TYPE | UNIQUENESS | COLUMNS                                     |
|------------|------------|------------|---------------------------------------------|
| SALES_IX01 | NORMAL     | UNIQUE     | PROD_ID+CUST_ID+TIME_ID+CHANNEL_ID+PROMO_ID |
| SALES_IX02 | NORMAL     | NONUNIQUE  | CUST_ID                                     |
| SALES_IX03 | NORMAL     | NONUNIQUE  | TIME_ID                                     |
| SALES_IX04 | NORMAL     | NONUNIQUE  | CHANNEL_ID                                  |
| SALES_IX05 | NORMAL     | NONUNIQUE  | PROMO_ID                                    |

해결)

1. 수행에 필요한 컬럼만으로 인덱스를 생성하여 사용합니다.

```
SQL> CREATE INDEX sales_ix06 ON sales(channel_id,time_id) nologging ;
```

Index created.

```
SQL> SELECT channel_id, MAX(time_id)
 FROM sales
 WHERE channel_id = 3
 GROUP BY channel_id ;
```

```
SQL> @xplan
```

| Id  | Operation            | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT     |            | 1      |        | 1      | 00:00:00.15 | 1648    |
| 1   | SORT GROUP BY NOSORT |            | 1      | 1      | 1      | 00:00:00.15 | 1648    |
| * 2 | INDEX RANGE SCAN     | SALES_IX06 | 1      | 1      | 536K   | 00:00:00.06 | 1648    |

Predicate Information (identified by operation id):

```
2 - filter("CHANNEL_ID">=3)
```

인덱스로 필요한 범위만 스캔 후 결과를 만들어서 I/O가 좀 더 줄었습니다. 그러나 여전히 인덱스의 스캔 범위가 넓습니다. 만약 조건에 만족하는 행이 많으면 더 넓은 범위를 액세스하며 성능이 저하될 것입니다.



## 2. 인덱스 사용 및 ROWNUM 활용

```
SQL> SELECT /*+ index_rs_desc(sales sales_ix06) */ channel_id, time_id
 FROM sales
 WHERE channel_id = 3
 AND ROWNUM = 1;
```

```
CHANNEL_ID TIME_ID

 3 01/12/31
```

1 row selected.

```
SQL> @xplan
```

| Id  | Operation                   | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|-----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT            |            | 1      |        | 1      | 00:00:00.01 | 3       |
| * 1 | COUNT STOPKEY               |            | 1      |        | 1      | 00:00:00.01 | 3       |
| * 2 | INDEX RANGE SCAN DESCENDING | SALES_IX06 | 1      | 1      | 1      | 00:00:00.01 | 3       |

Predicate Information (identified by operation id):

```
1 - filter(ROWNUM=1)
2 - access("CHANNEL_ID"=3)
```

동일한 CHANNEL\_ID 값의 개수가 증가하여도 항상 일정한 I/O로 결과를 검색할 수 있으나 해당 인덱스의 사용이 불가능한 경우가 발생하면 잘못된 결과를 검색할 수 있습니다. 또한 ROWNUM의 조건식을 Subquery 안에서 사용하면 Query Transformation이 진행되지 못하는 경우도 발생하므로 전체 문장의 최적화가 힘들 수 있습니다.

## 3. 바인드 변수와 함께 인덱스를 활용한 MIN, MAX Operation의 사용

```
SQL> SELECT MAX(time_id)
 FROM sales
 WHERE channel_id = 3 ;
```

```
MAX(TIME
```

```

01/12/31
```

```
SQL> @xplan
```

| Id  | Operation                  | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT           |            | 1      |        | 1      | 00:00:00.01 | 3       |
| 1   | SORT AGGREGATE             |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| 2   | FIRST ROW                  |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| * 3 | INDEX RANGE SCAN (MIN/MAX) | SALES_IX06 | 1      | 1      | 1      | 00:00:00.01 | 3       |

```
Predicate Information (identified by operation id):
```

```
3 - access("CHANNEL_ID"=3)
```

```
SQL> VARIABLE channel NUMBER
```

```
SQL> EXECUTE :channel := 3 ;
```

```
SQL> SELECT :channel channel_id, MAX(time_id)
 FROM sales
 WHERE channel_id = :channel ;
```

```
CHANNEL_ID MAX(TIME
```

```

3 01/12/31
```

```
SQL> @xplan
```

| Id  | Operation                  | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT           |            | 1      |        | 1      | 00:00:00.01 | 3       |
| 1   | SORT AGGREGATE             |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| 2   | FIRST ROW                  |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| * 3 | INDEX RANGE SCAN (MIN/MAX) | SALES_IX06 | 1      | 1      | 1      | 00:00:00.01 | 3       |

```
Predicate Information (identified by operation id):
```

```
3 - access("CHANNEL_ID"=:CHANNEL)
```

## 4. 서브쿼리와 함께 인덱스를 활용한 MIN, MAX Operation의 사용

```
SQL> create table p_t
as
select distinct channel_id
from sales;
```

Table created.

```
SQL> select p.channel_id,
(select /*+ no_unnest */ max(time_id)
from sales
where sales.channel_id=p.channel_id) max_time_id
from p_t p
where channel_id=3;
```

CHANNEL\_ID MAX\_TIME

```

3 01/12/31
```

SQL> @xplan

| Id  | Operation                  | Name       | Starts | E-Rows | A-Rows | A-Time      | Buffers |
|-----|----------------------------|------------|--------|--------|--------|-------------|---------|
| 0   | SELECT STATEMENT           |            | 1      |        | 1      | 00:00:00.01 | 10      |
| 1   | SORT AGGREGATE             |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| 2   | FIRST ROW                  |            | 1      | 1      | 1      | 00:00:00.01 | 3       |
| * 3 | INDEX RANGE SCAN (MIN/MAX) | SALES_IX06 | 1      | 1      | 1      | 00:00:00.01 | 3       |
| * 4 | TABLE ACCESS FULL          | P_T        | 1      | 1      | 1      | 00:00:00.01 | 10      |

Predicate Information (identified by operation id):

```
3 - access("SALES"."CHANNEL_ID"=:B1)
4 - filter("CHANNEL_ID"=3)
```

-- 테이블 생성 대신 인라인 뷰 사용

```
drop table p_t;
```

```
select p.channel_id,
(select /*+ no_unnest */ max(time_id)
from sales
where sales.channel_id=p.channel_id) max_time_id
from (select distinct channel_id from sales) p
where channel_id=3;
```

```
SQL> DROP INDEX sales_ix06;
```